

S2(DS)_Decoder.cpp and L0toL1_S2(DS).cpp

Special Thanks to Ben Greenwood who's S2BGC c++ code is the founding basis for the S2(DS)_Decoder.cpp code.

How to compile: from the top directory "g++ ./src/L1/L0toL1_S2(DS).cpp"

This command is also in the makefile so should compile when running the makefile to compile the parser.

The use of this code follows Ben Greenwood's original S2BGC c++ parser. However there are a few modifications.

An environmental variable needs to be created...e.g.

```
export S2BGC_PATH="/data/Argo_Processing/S2BGC_Decoder-23Dec25"
```

```
export S2_PATH="/data/Argo_Processing/S2_Decoder"
```

```
export DS_PATH="/data/Argo_Processing/DS_Decoder"
```

For the L0 parser, one calls the routine with the float SN and cycle as the two inputs (e.g. "S2_Decoder 4019 8"). This implies to the code that cycle 8 from float 4019 will be run. For the L1 translator the code expects 2 inputs "L0toL1 4019 8". Here the first parameter is again the float serial number, while the 2nd is the cycle to begin the conversion. This cycle should already have an L1 file as it is the 'target' from which the 'Mission' is identified. The code will run all L1 from the target to the end of the hex files (highest cycle number). Alternatively if one only calls the routine with the float serial number, the L0toL1 code will run all cycles, starting at cycle -1 (where there should be a 'Mission' from the float startup).

The data directory contains 2 directories, hex and json. Sample hex data is included. Please note the Iridium position in the hex data (which is different then the S2BGC where there is no Iridium position). The S2(DS)_Decoder writes the output to ./data/json/L0. The L0toL1 code writes output to ./data/json/L1.

DS_Decoder.cpp can parse SBE61 float firmware V0.6-V2.2. This should include all active floats. S2_Decoder.cpp can parse SBE41 float firmware V2.0-V2.6 This should include all active floats.

I've included an example of S2 data (float 3383 V2.6) and 2 examples of DS data (6131 and 6141). Both the Deep examples are the latest float firmware V2.2. Float 6131 has a 2nd pressure sensor on board. To handle this extra sensor I've placed a float-specific "modified_6131_config.json", in its data directory. The code will default to this version of the config.json when this file is present. It is not required to be present (in which case it will read the ./config/config.json). This is one way that differences between floats can be handled.